SenseAI Ingest

Public datasets, where to store them, and how to get them into the pipeline

Version 1 · October 2026 · Companion to the senseai-institute/ingest repository

Who this is for. Someone who will set up the data and cloud accounts for the team, with no software engineering background assumed. Every command you need to type is shown verbatim; everything else is a click in a website. Where a step needs an engineer, it says so.

What you will end up with. (1) A cloud storage bucket holding the datasets we chose; (2) a rented computer that runs our ingest pipeline on them; (3) processed result files ready for the web viewer that will be deployed on Railway.

Contents

- 1. What the datasets are, in plain words
- 2. Download links and licences
- 3. Where to store the data (and why not on the web server)
- 4. Step-by-step setup: storage bucket, work computer, pipeline
- 5. Loading and checking each dataset
- 6. Connecting the results to the Railway viewer
- 7. Costs, safety and clean-up
- 8. Glossary

1. What the datasets are, in plain words

Our robot has several senses: event cameras, normal cameras, microphones, a motion sensor (IMU), and touch and motor sensors. Before we own that hardware, we test the pipeline on recordings that universities have published for free. Each dataset below was recorded by strapping sensors to a car, a drone, a walking robot or a person and driving or walking around.

Event camera. A camera that does not take pictures. Each pixel reports only when its brightness changes, up to millions of times per second. Fast and efficient, and the reason timing matters so much in this project. Prophesee and iniVation are the two main makers.

IMU. The motion sensor: an accelerometer plus a gyroscope, the same kind of chip that tells your phone it has been rotated.

LiDAR. A spinning laser that measures distances to build a 3-D picture of the surroundings. We do not decode it yet.

ROS bag file. A standard recording format used by most robots (from ROS, the Robot Operating System, the industry-standard messaging layer for robot software). Our pipeline reads these directly.

HDF5 file. A common file format for large scientific datasets, familiar from microscopy and genomics. Several of the datasets below use it, and our pipeline reads it.

Recommended starting basket

Under 20 GB in total. One recording from each of the first four, plus the small sound set.

Dataset	What it is	Why we want it
DSEC	Driving around Zurich, day and night, with event cameras, normal cameras, LiDAR and IMU.	Best-documented event dataset. Our primary test case.
M3ED	The same sensor rig on a car, a four-legged walking robot and a drone, indoors and out.	Closest match to a real robot carrying our whole sensor set. Already hosted on Amazon.
MVSEC	Older sibling of M3ED (cars, motorbike, indoor drone).	Small and simple; also tests our ROS bag reader.
EuRoC MAV	A small drone flying indoors with normal cameras and an IMU; millimetre-accurate ground truth.	The standard dataset for checking clock alignment. No event camera.
ESC-50	2,000 five-second clips of everyday sounds (dog bark, door knock, glass breaking).	First audio test material. Note the non-commercial licence.

Later additions: TUM-VIE (human-speed motion with event cameras), LOCATA (microphone arrays, for 'where is the sound coming from'), FSD50K (a much larger sound set), N-MNIST / N-Caltech101 (tiny recognition sets), Prophesee's automotive sets (very large; same sensor as the EVK4 camera we plan to buy).

2. Download links and licences

All links were checked in October 2026. Sizes are approximate. 'Research licence' means free for research and internal testing; check the terms before using the data in a shipped product.

Dataset	Download page	Size	Licence / catches
DSEC	https://dsec.ifi.uzh.ch/ Click 'Download', pick one sequence (e.g. zurich_city_04_a).	A few GB per sequence; ~1 TB total	Research licence. Download events, images, IMU for one sequence first.
M3ED	https://m3ed.io Amazon mirror: https://registry.opendata.aws/m3ed/	A few GB per recording; 100+ GB total	CC BY-SA 4.0 (commercial use allowed with attribution). No download needed if we use Amazon's copy.
MVSEC	https://daniilidis-group.github.io/mvsec/	1-10 GB per scene	Research licence. Offered as HDF5 and as ROS bag files; both work.
EuRoC MAV	https://projects.asl.ethz.ch/datasets/euroc-mav/ Bag-file links also listed at docs.ros.org (OpenVINS)	1-2 GB per flight	Research licence. Choose the '.bag' version.
TUM-VIE	https://cvg.cit.tum.de/data/datasets/visual-inertial-event-dataset	5-20 GB per sequence	Research licence. HDF5, same style as DSEC.
LOCATA	https://www.locata.lms.tf.fau.de/tasks/	A few GB	Requires registering with an email address.
ESC-50	https://github.com/karolpiczak/ESC-50	600 MB (one zip)	CC BY-NC: non-commercial. Fine for internal testing, not for a shipped product trained on it.
FSD50K	https://annotator.freesound.org/fsd/release/FSD50K/ (follow its Zenodo link)	~30 GB	Mostly CC-BY, but each clip carries its own licence.
N-MNIST / N-Caltech101	https://www.garrickorchard.com/datasets/n-mnist https://www.garrickorchard.com/datasets/n-caltech101	Under 1 GB each	Free; manual download only.
Prophesee Gen1 / 1-Megapixel	https://prophesee.ai/2020/01/24/prophesee-gen1-automotive-detection-dataset/ prophesee.ai dataset page	Hundreds of GB	Must accept Prophesee's terms. Torrent download, reported as unreliable. Skip for now.

3. Where to store the data

Think of three different rooms. The raw datasets go in the warehouse; the work happens on the lab bench; only small, finished results go in the shop window.

Room	Service	What goes there	Rough cost
Warehouse	Amazon S3 (a 'bucket')	The raw datasets. Permanent, bottomless, readable by anything we run.	About $0.023 per GB per month: 20 GB is roughly 50 cents a month.
Lab bench	One Amazon EC2 computer	Where the pipeline runs. Switched on when working, off when not.	About $0.08 per hour for a t3.large while on; nothing while off (plus a few dollars a month for its disk).
Shop window	Railway	The web viewer. Reads only small result files, never the raw datasets.	About $5-20 per month.

Why not keep the data on the EC2 computer? Its disk is tied to that one machine, costs more per GB, and disappears if the machine is deleted. S3 is cheaper, permanent, and the viewer and any future machine can read from it.

Why not put the data on Railway? Railway is built for small web apps. Its storage is limited and comparatively expensive, and uploading multi-gigabyte files there is slow. The viewer only needs the compact result files our pipeline produces (megabytes, not gigabytes).

Which region? Choose **US West (Oregon)**, called *us-west-2*, for everything. M3ED's public copy lives there, so reading it is free and fast. Keeping the bucket and the computer in the same region also avoids data-transfer charges between them.

Data flow: warehouse (S3) → lab bench (EC2 runs senseai-ingest) → small result files back to S3 → shop window (Railway viewer reads them).

4. Step-by-step setup

Total time about an hour, most of it waiting. You need a credit card for Amazon and a GitHub login that can see the senseai-institute/ingest repository.

A. Create the Amazon account and the storage bucket (about 20 minutes)

1. Go to https://aws.amazon.com and create an account (or sign in). Turn on two-factor authentication when offered.
2. In the search box at the top, type **S3** and open it.
3. Click **Create bucket**. Name: **senseai-datasets** (bucket names must be unique worldwide, so add your company suffix if that is taken, e.g. senseai-datasets-acme). Region: **US West (Oregon) us-west-2**. Leave every other setting at its default, which keeps the bucket private. Click **Create bucket**.
4. Inside the bucket, click **Create folder** three times to make **raw**, **processed** and **viewer**. (Raw datasets, pipeline outputs, and the small files the viewer reads.)

B. Create the work computer (about 30 minutes)

1. In the AWS search box type **EC2** and open it. Make sure the region shown in the top right is **Oregon**.
2. Click **Launch instance**. Name: **senseai-workbench**. Image: **Ubuntu Server 24.04 LTS**. Instance type: **t3.large** (2 CPUs, 8 GB memory; plenty for replaying datasets).
3. Key pair: click **Create new key pair**, name it senseai-workbench, download the file and keep it somewhere safe (it is the key to the machine; it cannot be re-downloaded).
4. Network settings: keep the defaults (allow SSH from 'My IP' is a sensible choice).

5. Storage: set **100 GiB**, type gp3.
6. Expand **Advanced details**, find **IAM instance profile** and click 'Create new IAM profile'. In the page that opens, create a role for EC2 with the policy **AmazonS3FullAccess** and name it senseai-workbench-role. Back in the launch page, refresh the list and select it. This is what lets the computer read and write your bucket without any passwords in the code.
7. Click **Launch instance**. After about a minute it shows as 'Running'.
8. Select it and click **Connect** → **EC2 Instance Connect** → **Connect**. A black terminal window opens in your browser. Everything below is typed into that window.

C. Install the pipeline on the work computer (about 10 minutes)

Paste these lines one block at a time and press Enter. Lines starting with # are comments and can be pasted too.

```
# 1. Basic tools
sudo apt-get update && sudo apt-get install -y git make unzip aria2 curl
curl -LsSf https://astral.sh/uv/install.sh | sh && source ~/.bashrc
curl 'https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip' -o awscliv2.zip
unzip -q awscliv2.zip && sudo ./aws/install

# 2. Our code (you will be asked for your GitHub login the first time)
git clone https://github.com/senseai-institute/ingest.git
cd ingest && uv sync --extra dev && make test
```

The last command runs the automated test suite. The final lines should say something like *132 passed*. If it does not, stop and send the output to the engineer.

```
# 3. Check the computer can see the bucket (replace the name if you changed it)
aws s3 ls s3://senseai-datasets/
```

You should see the three folders you created. If you see 'Access Denied', the IAM role from step B6 is missing.

5. Loading and checking each dataset

The pattern is always the same three commands: **get** the data, **inspect** it (does our pipeline understand the file?), **replay** it (produce the aligned output and quality report). Everything is downloaded on the work computer, not on your laptop, so nothing large crosses your home connection.

First make a working folder:

```
mkdir -p ~/data && cd ~/data
```

DSEC (one sequence)

1. On https://dsec.ifi.uzh.ch/ open the download page and find the sequence **zurich_city_04_a**. Right-click each of its links (events left, events right, images, IMU) and choose 'Copy link address'.
2. On the work computer, download and unpack each one into a folder with the sequence name:

```
mkdir -p dsec/zurich_city_04_a && cd dsec/zurich_city_04_a
aria2c -x 8 '<paste the events-left link here>'
aria2c -x 8 '<paste the events-right link>'
aria2c -x 8 '<paste the images link>'
unzip -q '*.zip' && cd ~/data
```

1. Check and replay:

```
cd ~/ingest
uv run senseai-ingest inspect --path ~/data/dsec/zurich_city_04_a
uv run senseai-ingest replay --dataset dsec --path ~/data/dsec/zurich_city_04_a \
    --tick-ms 10 --speed 0 --out ~/data/processed/dsec_zurich_city_04_a.h5 --stats
```

'inspect' should print *layout: dsec* and a list of sources. 'replay' prints a small table: ticks produced, timing jitter (should be well under 1 ms), drops (should be 0) and the rate of each sensor. If 'inspect' prints *unknown*, the file layout differs from what we expect; send the printed output to the engineer, it is usually a one-line fix.

M3ED (no download; read from Amazon's copy)

```
# List what is available (no account needed)
aws s3 ls --no-sign-request s3://m3ed-dist/
# Copy one recording (pick any *_data.h5 you saw; this is an example name)
mkdir -p ~/data/m3ed && aws s3 cp --no-sign-request \
    s3://m3ed-dist/<folder>/<recording>_data.h5 ~/data/m3ed/
cd ~/ingest && uv run senseai-ingest inspect --path ~/data/m3ed/<recording>_data.h5
uv run senseai-ingest replay --dataset m3ed --path ~/data/m3ed/<recording>_data.h5 \
    --tick-ms 10 --speed 0 --out ~/data/processed/m3ed_<recording>.h5 --stats
```

Our M3ED reader was written from the published description and has not yet been checked against a real file. The 'inspect' step is exactly that check.

MVSEC (one scene)

1. On https://daniilidis-group.github.io/mvsec/ find **indoor_flying1** and copy the link to the HDF5 data file (named like indoor_flying1_data.hdf5).
2. Download and replay:

```
mkdir -p ~/data/mvsec && cd ~/data/mvsec && aria2c -x 8 '<paste link>' && cd ~/ingest
uv run senseai-ingest inspect --path ~/data/mvsec/indoor_flying1_data.hdf5
uv run senseai-ingest replay --dataset mvsec --path ~/data/mvsec/indoor_flying1_data.hdf5 \
    --tick-ms 10 --speed 0 --out ~/data/processed/mvsec_indoor_flying1.h5 --stats
```

EuRoC MAV (one flight, ROS bag format)

1. On https://projects.asl.ethz.ch/datasets/euroc-mav/ (or the OpenVINS list at docs.ros.org) copy the link to the **.bag** file of **Machine Hall 01** (MH_01_easy.bag).
2. Download and replay (note: --dataset rosbag):

```
mkdir -p ~/data/euroc && cd ~/data/euroc && aria2c -x 8 '<paste link>' && cd ~/ingest
uv run senseai-ingest replay --dataset rosbag --path ~/data/euroc/MH_01_easy.bag \
    --tick-ms 10 --speed 0 --out ~/data/processed/euroc_MH_01.h5 --stats
```

ESC-50 (sounds)

```
mkdir -p ~/data/esc50 && cd ~/data/esc50
curl -L -o esc50.zip https://github.com/karoldvl/ESC-50/archive/master.zip && unzip -q
esc50.zip
cd ~/ingest && uv run senseai-ingest replay --dataset wav \
    --path ~/data/esc50/ESC-50-master/audio/1-100032-A-0.wav --out
~/data/processed/esc50_sample.h5 --stats
```

Copy everything to the warehouse

Raw downloads go to **raw/**, pipeline outputs to **processed/**. This can be repeated safely; it only uploads what changed.

```
aws s3 sync ~/data/dsec s3://senseai-datasets/raw/dsec
aws s3 sync ~/data/mvsec s3://senseai-datasets/raw/mvsec
aws s3 sync ~/data/euroc s3://senseai-datasets/raw/euroc
aws s3 sync ~/data/esc50 s3://senseai-datasets/raw/esc50
aws s3 sync ~/data/processed s3://senseai-datasets/processed
```

Then **stop** the work computer (EC2 → select it → Instance state → Stop) so it stops costing money. Everything is safe in the bucket; the machine can be started again any time.

6. Connecting the results to the Railway viewer

The viewer is a separate, small web application (its own GitHub repository, built with the Claude Code prompt that accompanies this guide). It never touches the raw datasets. It reads a compact 'summary' of each processed recording from the **viewer/** folder of the bucket and draws timelines, frame thumbnails, sensor traces and the quality report in the browser.

How the pieces connect

1. The pipeline's **replay** command writes a processed file (tens to hundreds of MB).
2. A small **export** step in the viewer repository turns each processed file into a summary (a few MB: per-slice counts, downsampled sensor traces, a handful of JPEG thumbnails, the stats) and uploads it to **s3://senseai-datasets/viewer/<recording>/**. It runs on the work computer with one command: *uv run senseai-viewer export ~/data/processed/dsec_zurich_city_04_a.h5*.
3. The viewer app on Railway lists whatever is in the viewer/ folder and shows it. New recordings appear with no redeploy.

Deploying the viewer on Railway (about 15 minutes, once)

1. Create a Railway account at https://railway.app and link it to GitHub.
2. Click **New Project** → **Deploy from GitHub repo** → choose the viewer repository. Railway detects the app and builds it.
3. Open the service's **Variables** tab and add: **S3_BUCKET** = senseai-datasets, **AWS_REGION** = us-west-2, **AWS_ACCESS_KEY_ID** and **AWS_SECRET_ACCESS_KEY** (see below), and **VIEWER_PASSWORD** = a shared password for the team.
4. In **Settings** → **Networking** click **Generate domain**. That URL is the viewer. Railway rebuilds automatically whenever the viewer code changes on GitHub.

Making the read-only key for Railway

Railway sits outside Amazon, so it needs a key. Give it one that can only *read* the viewer folder, nothing else:

1. In AWS, open **IAM** → **Users** → **Create user**, name it **railway-viewer**.
2. Attach a policy with **Create policy** → **JSON** and paste the block below (change the bucket name if yours differs).
3. Open the user → **Security credentials** → **Create access key** → 'Application running outside AWS'. Copy the two values into Railway's Variables. They are shown only once.

```
{ "Version": "2012-10-17", "Statement": [
  { "Effect": "Allow", "Action": ["s3:ListBucket"],
    "Resource": "arn:aws:s3:::senseai-datasets",
    "Condition": { "StringLike": { "s3:prefix": ["viewer/*"] } } },
  { "Effect": "Allow", "Action": ["s3:GetObject"],
    "Resource": "arn:aws:s3:::senseai-datasets/viewer/*" } ] }
```

7. Costs, safety and clean-up

Item	Typical monthly cost for the starting basket
S3 storage, ~20 GB raw + results	under $1
EC2 t3.large, used ~20 hours/month, 100 GB disk	about $2 for compute + about $8 for the disk
Railway viewer	$5-20

Item	Typical monthly cost for the starting basket
Data transfer	Negligible while bucket, computer and M3ED's copy are all in Oregon

- **Always stop the EC2 computer** when not in use. A forgotten t3.large costs about $60 a month.
- Set an AWS **Budget** (search 'Budgets' in AWS) with an email alert at, say, $50 a month.
- Keep the bucket private (the default). The viewer reads it through its own limited key; nothing is public.
- Never paste the Railway access key anywhere except Railway's Variables page. If it leaks, delete it in IAM and create a new one.
- **Licences:** ESC-50 is non-commercial. DSEC, MVSEC, EuRoC and TUM-VIE are research licences. M3ED (CC BY-SA) and most of FSD50K (CC-BY) allow commercial use with attribution. Review before any of this data touches a product.
- To delete everything: empty and delete the bucket, terminate the EC2 instance, delete the Railway project.

8. Glossary

Term	Meaning
AWS	Amazon Web Services: Amazon's rentable computers and storage.
S3 / bucket	Amazon's file storage. A bucket is one top-level folder with its own name and permissions.
EC2 / instance	A rented virtual computer. 'Instance' is Amazon's word for one of them.
IAM / role / key	AWS's permissions system. A role lets a machine act without a password; a key is a password-like pair for software running elsewhere (e.g. Railway).
Region	Which of Amazon's data centres hosts your things. We use Oregon (us-west-2).
Railway	A service that hosts small web apps straight from a GitHub repository.
GitHub repository	Where the code lives and is version-controlled.
Terminal	The text window where commands are typed. EC2 Instance Connect opens one in the browser.
uv	The tool that installs and runs our Python code. 'uv run ...' means 'run this with the right setup'.
Tick	One 10-millisecond slice of aligned sensor data, the unit the AI models consume.
Event camera	A camera whose pixels report brightness changes rather than taking frames.
IMU	Motion sensor (accelerometer + gyroscope).
LiDAR	Laser distance scanner.
HDF5	A file format for large scientific datasets.
ROS / ROS 2 / bag file	The standard robot-software messaging system and its recording format.
Jitter	How far off the timing alignment is. Our target is under 1 ms in 99% of ticks.